

Speculative decoding on Apertus-v1.5-70B: the 8B drafter against n-gram

Six independently launched deployments on 4×GH200 Clariden nodes.

Same corpus, same client, same max_model_len. Three workloads × two concurrencies × six deployments = 36 cells, all at a 1.000 success rate.

jobs 3391425 / 3405008 / 3391426 / 3392153 / 3392370 / 3403354

depth 3 · draft TP=4 · n-gram lookup 1-4

2 deployment repeats per arm

smoke corpus, 6 prompts

Verdict

DO NOT SHIP

Do not enable draft_model speculation for Apertus 1.5 70B on this stack

The 8B drafter accepts 60–80% of its proposed tokens and still returns 0.375–0.472× baseline output throughput in all twelve of its cells, across two independent deployments. The cost is not acceptance and not verification. A speculative step with the drafter costs 6.2–7.5× a baseline decode step, of which the drafter alone is 4.9–6.2×. Three 8B forward passes at TP=4 cannot be five to six whole 70B steps, so that is per-step overhead in the draft path.

CARRY FORWARD

N-gram is the only positive result, and it survives every denominator

It accepts only 13–29%. On long-context summarization it beats baseline in both of its deployments and against both baselines: 1.127–1.204× at concurrency 1 and 1.071–1.098× at concurrency 8. No ratio in the study moves by more than 0.005 when the denominator is swapped. All six n-gram cells clear the 10% spread gate, worst 6.6%. Faster to first token in 22 of 24 comparisons.

Acceptance is not the binding constraint. The arm that accepts worst is the only arm that wins.

Headline numbers

0.375-0.472×

draft-n3-tp4 output throughput vs baseline (12 cells)

0.848-1.204×

ngram-n3 output throughput vs baseline (12 cells)

1.8-2.7×

n-gram over draft repeat 1, every cell

6.2-7.5×

draft engine step vs baseline decode step (derived)

1.23-1.36×

verification + scheduling change only (n-gram)

−51%

KV-cache tokens after hosting the 8B drafter

Baseline itself reproduces to 0.41% on output throughput across two nodes. Speculative arms spread 1.7–11.1% (draft) and 1.7–6.6% (n-gram). Speculation costs predictability as well as capacity.

N-gram vs draft vs baseline

Output tokens per second. Ratios divide by baseline repeat 1; swapping to repeat 2 or the mean moves no ratio by more than 0.005. Bold green cells beat baseline.

WORKLOAD	CONC.	BASELINE TOK/S	DRAFT R1 → R2	N-GRAM R1 → R2	D÷B R1 / R2	N÷B R1 / R2
open_chat	1	70.1	31.0 → 29.1	63.3 → 64.3	0.442 / 0.415	0.902 / 0.917
open_chat	8	432.3	190.6 → 173.2	366.5 → 372.9	0.441 / 0.401	0.848 / 0.863
code	1	70.5	33.0 → 32.5	66.5 → 69.1	0.469 / 0.461	0.943 / 0.980
code	8	540.6	255.2 → 238.5	464.3 → 480.3	0.472 / 0.441	0.859 / 0.888
long_context_summarization	1	70.2	29.4 → 26.3	79.2 → 84.5	0.419 / 0.375	1.127 / 1.204
long_context_summarization	8	493.7	212.0 → 200.8	528.7 → 539.8	0.429 / 0.407	1.071 / 1.093

DRAFT ACCEPTANCE

0.605-0.797 · mean length 2.81-3.39 / 4

Near-invariant between deployments (0.5–4.3%). Code accepts best. Quality is a property of the model pair, not the node.

N-GRAM ACCEPTANCE

0.127-0.285 · mean length 1.38-1.85 / 4

Workload-dependent in the prompt-lookup direction: summarization accepts best because a summary quotes the document. Code generation has little to copy forward. Moves 2.0–13.9% between deployments.

Where the speculative step cost goes

Per-token latency understates per-step cost. Mean step time is derived as $\text{mean TPOT} \times T \div (T - \text{accepted})$. The n-gram arm proposes with no forward pass, so it isolates verification.

~14.2 ms

baseline decode step (TPOT = step time)

17.4-20.0 ms

n-gram step = verify + scheduling · 1.23-1.36×

91-106 ms

draft step · extra 72-88 ms · 6.2-7.5× total

WORKLOAD	CONC.	BASELINE STEP	N-GRAM R1 / R2	DRAFT R1 / R2	VERIFY ×	DRAFT TOTAL ×
open_chat	1	14.13 ms	18.64 / 18.32	91.15 / 100.41	1.32 / 1.30×	6.45 / 7.11×
open_chat	8	14.67 ms	20.00 / 19.61	94.24 / 101.27	1.36 / 1.34×	6.42 / 6.90×
code	1	14.15 ms	18.13 / 17.45	101.62 / 104.25	1.28 / 1.23×	7.18 / 7.37×
code	8	14.70 ms	19.54 / 18.88	101.34 / 106.45	1.33 / 1.28×	6.89 / 7.24×
long_context_summarization	1	14.16 ms	17.79 / 17.36	96.63 / 105.52	1.26 / 1.23×	6.82 / 7.45×
long_context_summarization	8	14.71 ms	19.42 / 18.98	91.77 / 98.56	1.32 / 1.29×	6.24 / 6.70×

Two n-gram deployments on different nodes put step cost within 1.7-3.9% of each other. A node effect cannot absorb a drafter cost of five to six baseline steps. KV cache: baseline 513,696 tokens; n-gram -18.9% with no drafter resident; draft -51.0%.

What this run cannot claim

Two serving properties are confounded with method

Async scheduling is auto-disabled for both speculative arms, not the baseline. The per-step token budget shrinks to 7168 for draft_model only (baseline and n-gram stay at 8192), so part of the 72-88 ms is budget, not drafter.

The smoke “long context” is ~283 prompt tokens

Not 16k-64k. These rows measure almost no prefill. N-gram’s best regime is untested — this probably understates its advantage.

The draft arm ran a patched image

Stock vLLM crashes at drafter load (image_token_index). Overlay is swiss-ai/vllm#20, unmerged. Baseline and n-gram used the stock image.

Do not present “the gate passes for n-gram”

Six greedy prompts. Two n-gram deployments return opposite calibrated-gate verdicts. Divergence between arms is the same size as divergence between two deployments of one arm — including two baselines (1 of 6 exact).

One draft cell breaches the 10% spread gate

long_context_summarization at concurrency 1 moves 11.1% between draft deployments. Direction is unaffected. All six n-gram cells pass, worst 6.6%.

Not measured

Concurrency 32, depths other than 3, draft TP=1 (unservable), confirmation-matrix bootstrap CIs, GPU / power telemetry, a within-deployment correctness capture.

The ask for this room

Profile the draft path. Find where 72–88 ms per step goes when three 8B forward passes should cost a fraction of one 70B step.

- Highest-value next experiment, and a vLLM question: captured graphs, model-switch / sync overhead, 7168-token budget.
- If that overhead is fixable, the whole conclusion changes.
- Also: a genuinely long-context summarization workload; a baseline with async scheduling off; a path for `swiss-ai/vllm#20` into a built image.

Numbers from `results/smoke-screening-20260913-debug/README.md` (preferred over `docs/hackathon-20260915.md` where they disagree: this deck uses six deployments / 36 cells, the baseline repeat, and TTFT 22 of 24). Companion canvas: `canvases/apertus-speculative-decoding-smoke-screening.canvas.tsx`.